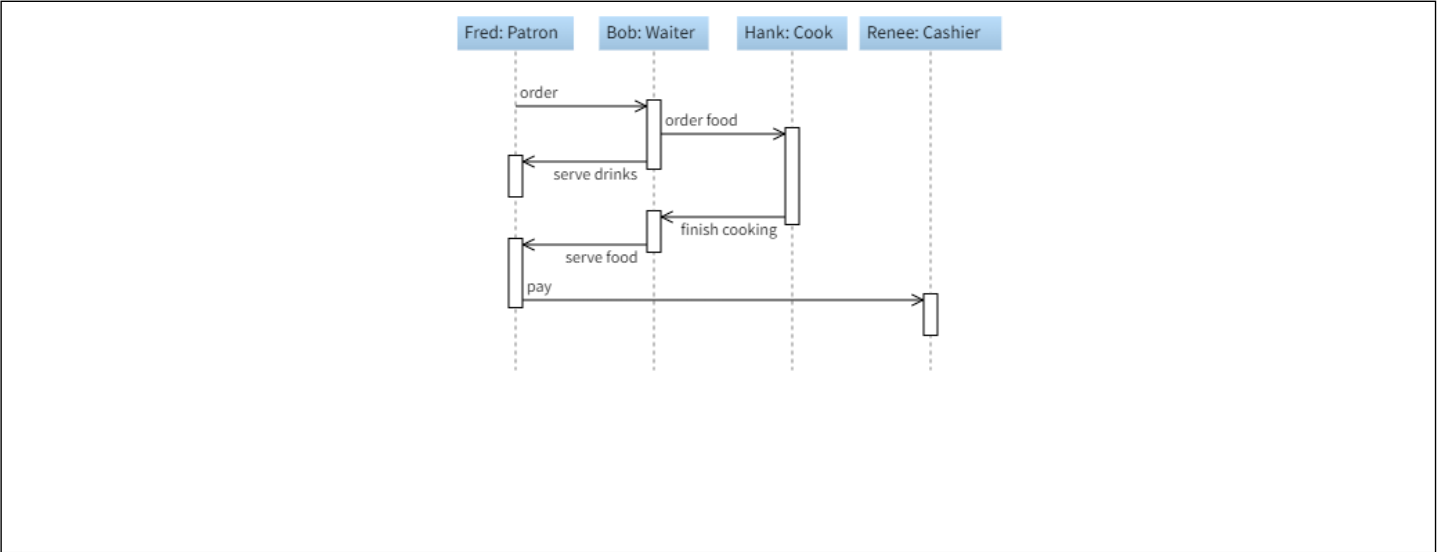




A *sequence diagram* is an interaction diagram that shows how entities operate with one another and in what order. In this sample, we show the interaction between different people in a restaurant.

The diagram uses the [Diagram.groupTemplate](#) for "lifelines," [Diagram.nodeTemplate](#) for "activities," and [Diagram.linkTemplate](#) for "messages" between the entities. Also featured are a custom Link class and custom [LinkingTool](#) to draw links between lifelines and create activities at the end of the new link. Nodes use a binding function on the location property to ensure they are anchored to their lifeline.

Save Load Diagram Model saved in JSON format:

```
{ "class": "go.GraphLinksModel",
  "nodeDataArray": [
    { "key": "Fred", "text": "Fred: Patron", "isGroup": true, "loc": "0 0", "duration": 9 },
    { "key": "Bob", "text": "Bob: Waiter", "isGroup": true, "loc": "100 0", "duration": 9 },
    { "key": "Hank", "text": "Hank: Cook", "isGroup": true, "loc": "200 0", "duration": 9 },
    { "key": "Renee", "text": "Renee: Cashier", "isGroup": true, "loc": "300 0", "duration": 9 },
    { "group": "Bob", "start": 1, "duration": 2 },
    { "group": "Hank", "start": 2, "duration": 3 },
    { "group": "Fred", "start": 3, "duration": 1 },
    { "group": "Bob", "start": 5, "duration": 1 },
    { "group": "Fred", "start": 6, "duration": 2 },
```

GoJS version 2.3.6. Copyright 1998-2023 by Northwoods Software.

[View this sample page's source on GitHub](#)

View the code for this sample in-page

Download the HTML and JS to use as a starting point

```
function init() {

  // Since 2.2 you can also author concise templates with method chaining instead of GraphObject.make
  // For details, see https://gojs.net/latest/intro/buildingObjects.html
  const $ = go.GraphObject.make;

  myDiagram =
    $(go.Diagram, "myDiagramDiv", // must be the ID or reference to an HTML DIV
      {
        allowCopy: false,
        linkingTool: $(MessagingTool), // defined below
        "resizingTool.isGridSnapEnabled": true,
        draggingTool: $(MessageDraggingTool), // defined below
        "draggingTool.gridSnapCellSize": new go.Size(1, MessageSpacing / 4),
        "draggingTool.isGridSnapEnabled": true,
        // automatically extend Lifelines as Activities are moved or resized
      }
    );
```

```

        "SelectionMoved": ensureLifelineHeights,
        "PartResized": ensureLifelineHeights,
        "undoManager.isEnabled": true
    });

    // when the document is modified, add a "*" to the title and enable the "Save" button
    myDiagram.addDiagramListener("Modified", e => {
        const button = document.getElementById("SaveButton");
        if (button) button.disabled = !myDiagram.isModified;
        const idx = document.title.indexOf("*");
        if (myDiagram.isModified) {
            if (idx < 0) document.title += "*";
        } else {
            if (idx >= 0) document.title = document.title.slice(0, idx);
        }
    });

    // define the Lifeline Node template.
    myDiagram.groupTemplate =
        $(go.Group, "Vertical",
            {
                locationSpot: go.Spot.Bottom,
                locationObjectName: "HEADER",
                minLocation: new go.Point(0, 0),
                maxLocation: new go.Point(9999, 0),
                selectionObjectName: "HEADER"
            },
            new go.Binding("location", "loc", go.Point.parse).makeTwoWay(go.Point.stringify),
            $(go.Panel, "Auto",
                { name: "HEADER" },
                $(go.Shape, "Rectangle",
                    {
                        fill: $(go.Brush, "Linear", { 0: "#bbdefb", 1: go.Brush.darkenBy("#bbdefb", 0.1) }),
                        stroke: null
                    },
                ),
                $(go.TextBlock,
                    {
                        margin: 5,
                        font: "400 10pt Source Sans Pro, sans-serif"
                    },
                    new go.Binding("text", "text"))
            ),
            $(go.Shape,
                {
                    figure: "LineV",
                    fill: null,
                    stroke: "gray",
                    strokeDashArray: [3, 3],
                    width: 1,
                    alignment: go.Spot.Center,
                    portId: "",
                    fromLinkable: true,
                    fromLinkableDuplicates: true,
                    toLinkable: true,
                    toLinkableDuplicates: true,
                    cursor: "pointer"
                },
                new go.Binding("height", "duration", computeLifelineHeight))
        );

    // define the Activity Node template
    myDiagram.nodeTemplate =
        $(go.Node,
            {
                locationSpot: go.Spot.Top,
                locationObjectName: "SHAPE",
                minLocation: new go.Point(NaN, LinePrefix - ActivityStart),
                maxLocation: new go.Point(NaN, 19999),
                selectionObjectName: "SHAPE",
                resizable: true,
                resizeObjectName: "SHAPE",
                resizeAdornmentTemplate:

```

```

        $(go.Adornment, "Spot",
            $(go.Placeholder),
            $(go.Shape, // only a bottom resize handle
                {
                    alignment: go.Spot.Bottom, cursor: "col-resize",
                    desiredSize: new go.Size(6, 6), fill: "yellow"
                })
        )
    },
    new go.Binding("location", "", computeActivityLocation).makeTwoWay(backComputeActivityLocation),
    $(go.Shape, "Rectangle",
        {
            name: "SHAPE",
            fill: "white", stroke: "black",
            width: ActivityWidth,
            // allow Activities to be resized down to 1/4 of a time unit
            minSize: new go.Size(ActivityWidth, computeActivityHeight(0.25))
        },
        new go.Binding("height", "duration", computeActivityHeight).makeTwoWay(backComputeActivityHeight))
    );

// define the Message Link template.
myDiagram.linkTemplate =
    $(MessageLink, // defined below
        { selectionAdorned: true, curviness: 0 },
        $(go.Shape, "Rectangle",
            { stroke: "black" }),
        $(go.Shape,
            { toArrow: "OpenTriangle", stroke: "black" }),
        $(go.TextBlock,
            {
                font: "400 9pt Source Sans Pro, sans-serif",
                segmentIndex: 0,
                segmentOffset: new go.Point(NaN, NaN),
                isMultiline: false,
                editable: true
            },
            new go.Binding("text", "text").makeTwoWay())
    );

// create the graph by reading the JSON data saved in "mySavedModel" textarea element
load();
}

function ensureLifelineHeights(e) {
    // iterate over all Activities (ignore Groups)
    const arr = myDiagram.model.nodeDataArray;
    let max = -1;
    for (let i = 0; i < arr.length; i++) {
        const act = arr[i];
        if (act.isGroup) continue;
        max = Math.max(max, act.start + act.duration);
    }
    if (max > 0) {
        // now iterate over only Groups
        for (let i = 0; i < arr.length; i++) {
            const gr = arr[i];
            if (!gr.isGroup) continue;
            if (max > gr.duration) { // this only extends, never shrinks
                myDiagram.model.setDataProperty(gr, "duration", max);
            }
        }
    }
}

// some parameters
const LinePrefix = 20; // vertical starting point in document for all Messages and Activations
const LineSuffix = 30; // vertical length beyond the last message time
const MessageSpacing = 20; // vertical distance between Messages at different steps
const ActivityWidth = 10; // width of each vertical activity bar
const ActivityStart = 5; // height before start message time
const ActivityEnd = 5; // height beyond end message time

```

```

function computeLifelineHeight(duration) {
  return LinePrefix + duration * MessageSpacing + LineSuffix;
}

function computeActivityLocation(act) {
  const groupdata = myDiagram.model.findNodeDataForKey(act.group);
  if (groupdata === null) return new go.Point();
  // get location of Lifeline's starting point
  const grouploc = go.Point.parse(groupdata.loc);
  return new go.Point(grouploc.x, convertTimeToY(act.start) - ActivityStart);
}

function backComputeActivityLocation(loc, act) {
  myDiagram.model.setDataProperty(act, "start", convertYToTime(loc.y + ActivityStart));
}

function computeActivityHeight(duration) {
  return ActivityStart + duration * MessageSpacing + ActivityEnd;
}

function backComputeActivityHeight(height) {
  return (height - ActivityStart - ActivityEnd) / MessageSpacing;
}

// time is just an abstract small non-negative integer
// here we map between an abstract time and a vertical position
function convertTimeToY(t) {
  return t * MessageSpacing + LinePrefix;
}

function convertYToTime(y) {
  return (y - LinePrefix) / MessageSpacing;
}

// a custom routed Link
class MessageLink extends go.Link {
  constructor() {
    super();
    this.time = 0; // use this "time" value when this is the temporaryLink
  }

  getLinkPoint(node, port, spot, from, ortho, othernode, otherport) {
    const p = port.getDocumentPoint(go.Spot.Center);
    const r = port.getDocumentBounds();
    const op = otherport.getDocumentPoint(go.Spot.Center);

    const data = this.data;
    const time = data !== null ? data.time : this.time; // if not bound, assume this has its own "time" property

    const aw = this.findActivityWidth(node, time);
    const x = (op.x > p.x ? p.x + aw / 2 : p.x - aw / 2);
    const y = convertTimeToY(time);
    return new go.Point(x, y);
  }

  findActivityWidth(node, time) {
    let aw = ActivityWidth;
    if (node instanceof go.Group) {
      // see if there is an Activity Node at this point -- if not, connect the link directly with the Group's lifeline
      if (!node.memberParts.any(mem => {
        const act = mem.data;
        return (act !== null && act.start <= time && time <= act.start + act.duration);
      })) {
        aw = 0;
      }
    }
    return aw;
  }

  getLinkDirection(node, port, linkpoint, spot, from, ortho, othernode, otherport) {
    const p = port.getDocumentPoint(go.Spot.Center);
    const op = otherport.getDocumentPoint(go.Spot.Center);
    const right = op.x > p.x;
  }
}

```

```

    return right ? 0 : 180;
}

computePoints() {
  if (this.fromNode === this.toNode) { // also handle a reflexive link as a simple orthogonal loop
    const data = this.data;
    const time = data !== null ? data.time : this.time; // if not bound, assume this has its own "time" property
    const p = this.fromNode.port.getDocumentPoint(go.Spot.Center);
    const aw = this.findActivityWidth(this.fromNode, time);

    const x = p.x + aw / 2;
    const y = convertTimeToY(time);
    this.clearPoints();
    this.addPoint(new go.Point(x, y));
    this.addPoint(new go.Point(x + 50, y));
    this.addPoint(new go.Point(x + 50, y + 5));
    this.addPoint(new go.Point(x, y + 5));
    return true;
  } else {
    return super.computePoints();
  }
}
}
// end MessageLink

// A custom LinkingTool that fixes the "time" (i.e. the Y coordinate)
// for both the temporaryLink and the actual newly created Link
class MessagingTool extends go.LinkingTool {
  constructor() {
    super();

    // Since 2.2 you can also author concise templates with method chaining instead of GraphObject.make
    // For details, see https://gojs.net/latest/intro/buildingObjects.html
    const $ = go.GraphObject.make;
    this.temporaryLink =
      $(MessageLink,
        $(go.Shape, "Rectangle",
          { stroke: "magenta", strokeWidth: 2 }),
        $(go.Shape,
          { toArrow: "OpenTriangle", stroke: "magenta" }));
  }

  doActivate() {
    super.doActivate();
    const time = convertYToTime(this.diagram.firstInput.documentPoint.y);
    this.temporaryLink.time = Math.ceil(time); // round up to an integer value
  }

  insertLink(fromnode, fromport, tonode, toport) {
    const newlink = super.insertLink(fromnode, fromport, tonode, toport);
    if (newlink !== null) {
      const model = this.diagram.model;
      // specify the time of the message
      const start = this.temporaryLink.time;
      const duration = 1;
      newlink.data.time = start;
      model.setDataProperty(newlink.data, "text", "msg");
      // and create a new Activity node data in the "to" group data
      const newact = {
        group: newlink.data.to,
        start: start,
        duration: duration
      };
      model.addNodeData(newact);
      // now make sure all Lifelines are long enough
      ensureLifelineHeights();
    }
    return newlink;
  }
}
// end MessagingTool

```

```

// A custom DraggingTool that supports dragging any number of MessageLinks up and down --
// changing their data.time value.
class MessageDraggingTool extends go.DraggingTool {
  // override the standard behavior to include all selected Links,
  // even if not connected with any selected Nodes
  computeEffectiveCollection(parts, options) {
    const result = super.computeEffectiveCollection(parts, options);
    // add a dummy Node so that the user can select only Links and move them all
    result.add(new go.Node(), new go.DraggingInfo(new go.Point()));
    // normally this method removes any links not connected to selected nodes;
    // we have to add them back so that they are included in the "parts" argument to moveParts
    parts.each(part => {
      if (part instanceof go.Link) {
        result.add(part, new go.DraggingInfo(part.getPoint(0).copy()));
      }
    })
    return result;
  }

  // override to allow dragging when the selection only includes Links
  mayMove() {
    return !this.diagram.isReadOnly && this.diagram.allowMove;
  }

  // override to move Links (which are all assumed to be MessageLinks) by
  // updating their Link.data.time property so that their link routes will
  // have the correct vertical position
  moveParts(parts, offset, check) {
    super.moveParts(parts, offset, check);
    const it = parts.iterator();
    while (it.next()) {
      if (it.key instanceof go.Link) {
        const link = it.key;
        const startY = it.value.point.y; // DraggingInfo.point.y
        let y = startY + offset.y; // determine new Y coordinate value for this link
        const cellY = this.gridSnapCellSize.height;
        y = Math.round(y / cellY) * cellY; // snap to multiple of gridSnapCellSize.height
        const t = Math.max(0, convertYToTime(y));
        link.diagram.model.set(link.data, "time", t);
        link.invalidateRoute();
      }
    }
  }
}
// end MessageDraggingTool

// Show the diagram's model in JSON format
function save() {
  document.getElementById("mySavedModel").value = myDiagram.model.toJson();
  myDiagram.isModified = false;
}
function load() {
  myDiagram.model = go.Model.fromJson(document.getElementById("mySavedModel").value);
}
window.addEventListener('DOMContentLoaded', init);

```